

Q.No:7

DS	Session	Tree 1	Question Information	Level 1	Challenge 67
Question description The sam is enjoying a wonderful vacation in Byteland. What makes the sam impressed the most is the road system of the country. Byteland has N cities numbered 1 through N. City 1 is the capital of Byteland. The country also has $N-1$ bidirectional roads connecting the cities. The i-th road connects two different cities u_i and v_i. In this road system, people can travel between every pair of different cities by going through exactly one path of roads. The roads are arranged in such a way that people can distinguish two cities only when both cities have different number of roads connected to it. Such two cities will be considered similar. For example, city A is similar to the capital if the number of roads connected to city A is equal to the number of roads connected to the capital. On each day during the vacation, the sam wants to have a trip as follows. He chooses two cities A and B such that the sam will visit city B if he goes from A to the capital using the shortest path. Then, the sam will visit the cities on the shortest path from A to B through this path. Please note that A may be equal to B; that means the sam will enjoy the day in a single city. The sam does not want to have similar trips. Two trips are considered similar if and only if they both have the same number of visited cities and for each i, the i-th city visited in one trip is similar to the i-th city visited in the other trip. The sam wants to have as many different, namely not similar, trips as possible. Help him count the maximum number of possible trips such that no two of them are similar. Input Format The first line of the input contains a single integer N. The i-th line of next $N-1$ lines contains two space-separated integers u_i and v_i, denoting the i-th road. Output Format Output a single line containing the maximum number of different trips.					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>3 2 1 3 1</pre> EXPECTED OUTPUT <pre>3</pre> Test Case 2 INPUT (STDIN) <pre>4 2 1 3 1 2 4</pre> EXPECTED OUTPUT <pre>5</pre>					

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005
#define MAXS 200010
#define MAXTR 700000
#define HASHSIZE 2097152

typedef long long ll;
```

```
struct State
{
    int len;
    int link;
    int head;
};

struct Transition
{
    int from;
    int to;
    int ch;
    int next;
};

struct State st[MAXS];
struct Transition tr[MAXTR];

int sz = 0;
int last = 0;
int trCnt = 0;
unsigned long long hkey[HASHSIZE];
int hval[HASHSIZE];
int head[MAXN];
int to[2 * MAXN];
int nextEdge[2 * MAXN];
int edgeCnt = 0;

int deg[MAXN];
ll dp[MAXS];

int order[MAXS];

unsigned long long makeKey(int state, int ch)
{
    return (((unsigned long long)(state + 1) << 32)
        | (unsigned int)ch);
}

unsigned long long hashKey(unsigned long long x)
{
    x ^= x >> 33;
    x *= 0xff51afd7ed558ccdULL;
    x ^= x >> 33;
    x *= 0xc4ceb9fe1a85ec53ULL;
    x ^= x >> 33;

    return x;
}

int findTransition(int state, int ch)
```

```
{
    unsigned long long key = makeKey(state, ch);
    unsigned long long pos = hashKey(key) & (HASHSIZE - 1);

    while (hval[pos] != 0)
    {
        if (hkey[pos] == key)
            return hval[pos] - 1;

        pos = (pos + 1) & (HASHSIZE - 1);
    }

    return -1;
}

void insertTransition(int index)
{
    unsigned long long key =
        makeKey(tr[index].from, tr[index].ch);

    unsigned long long pos =
        hashKey(key) & (HASHSIZE - 1);

    while (hval[pos] != 0)
    {
        if (hkey[pos] == key)
        {
            hval[pos] = index + 1;
            return;
        }

        pos = (pos + 1) & (HASHSIZE - 1);
    }

    hkey[pos] = key;
    hval[pos] = index + 1;
}

int addTransition(int from, int ch, int toState)
{
    int id = trCnt++;

    tr[id].from = from;
    tr[id].to = toState;
    tr[id].ch = ch;

    tr[id].next = st[from].head;
    st[from].head = id;

    insertTransition(id);
}
```

```
    return id;
}

void changeTransition(int state, int ch, int newTo)
{
    int id = findTransition(state, ch);

    if (id != -1)
        tr[id].to = newTo;
}

void sa_init()
{
    sz = 1;
    last = 0;

    st[0].len = 0;
    st[0].link = -1;
    st[0].head = -1;
}

void sa_extend(int c)
{
    int cur;
    int p;
    int q;
    int clone;
    int e;

    cur = sz++;

    st[cur].len = st[last].len + 1;
    st[cur].link = 0;
    st[cur].head = -1;

    p = last;

    while (p != -1)
    {
        e = findTransition(p, c);

        if (e != -1)
            break;

        addTransition(p, c, cur);

        p = st[p].link;
    }

    if (p == -1)
    {
```

```
    st[cur].link = 0;
}
else
{
    e = findTransition(p, c);
    q = tr[e].to;

    if (st[p].len + 1 == st[q].len)
    {
        st[cur].link = q;
    }
    else
    {
        clone = sz++;

        st[clone].len = st[p].len + 1;
        st[clone].link = st[q].link;
        st[clone].head = -1;
        e = st[q].head;

        while (e != -1)
        {
            addTransition(
                clone,
                tr[e].ch,
                tr[e].to
            );

            e = tr[e].next;
        }

        while (p != -1)
        {
            e = findTransition(p, c);

            if (e == -1 || tr[e].to != q)
                break;

            tr[e].to = clone;

            p = st[p].link;
        }

        st[q].link = clone;
        st[cur].link = clone;
    }
}

last = cur;
}
```

```
void addEdge(int u, int v)
{
    to[edgeCnt] = v;
    nextEdge[edgeCnt] = head[u];

    head[u] = edgeCnt;

    edgeCnt++;
}

void dfs(int s, int parent)
{
    int e;
    int tmp;

    sa_extend(deg[s]);

    tmp = last;

    for (e = head[s]; e != -1; e = nextEdge[e])
    {
        int child = to[e];

        if (child == parent)
            continue;

        dfs(child, s);

        last = tmp;
    }
}

int compareStates(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (st[x].len < st[y].len)
        return -1;

    if (st[x].len > st[y].len)
        return 1;

    return 0;
}

int main()
```

```
{
    int n;
    int i;
    int u, v;
    int e;

    printf("Input:\n");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
        head[i] = -1;

    for (i = 0; i < n - 1; i++)
    {
        scanf("%d %d", &u, &v);

        addEdge(u, v);
        addEdge(v, u);

        deg[u]++;
        deg[v]++;
    }

    sa_init();

    dfs(1, -1);
    for (i = 0; i < sz; i++)
        order[i] = i;

    qsort(
        order,
        sz,
        sizeof(int),
        compareStates
    );

    for (i = sz - 1; i >= 0; i--)
    {
        u = order[i];

        dp[u] = 1;

        e = st[u].head;

        while (e != -1)
        {
            dp[u] += dp[tr[e].to];

            e = tr[e].next;
        }
    }
}
```

```
}  
printf("Output:\n");  
  
printf("%lld\n", dp[0] - 1);  
  
return 0;  
}
```

Output:

Test_case 1:

```
Input:  
3  
2 1  
3 1  
Output:  
3
```

Test_case 2:

```
Input:  
4  
2 1  
3 1  
2 4  
Output:  
5
```